

Preparazione esame – UnityGameSDTRA

Domande prevedibili dal professore e dove modificare il codice — Sistemi per la Riabilitazione e la Terapia Assistita

Il professore tipicamente fa **modifiche live** per vedere se sai *dove* mettere le mani. Le domande qui sotto sono organizzate per difficoltà crescente, con il riferimento esatto al file e alla riga da modificare.

Livello 1 — Modifiche banali (quasi certe)

Sono quasi tutte in `Assets/Codice/Impostazioni.cs` — è il file da aprire per primo.

Domanda	Dove modificare
"Cambia il numero di vite a 5"	<code>VITE = 3 → 5</code>
"Dai 90 secondi per livello"	<code>TEMPO_LIVELLO = 60f → 90f</code>
"Una caramella deve valere 25 punti"	<code>PUNTI_CARAMELLA = 10 → 25</code>
"Quanti punti si perdono se tocchi un asteroide? Raddoppialo"	<code>PUNTI_PERSI_HIT = 5 → 10</code>
"Rendi più facile prendere le caramelle"	aumenta <code>RAGGIO_CARAMELLA</code>
"Nel livello 1 ci devono essere 20 caramelle"	<code>CARAMELLE_LIV1 = 10 → 20</code>
"Allunga il PRONTI iniziale a 3 secondi"	<code>TEMPO_PRONTI = 1.5f → 3f</code>

Trappola: se cambia solo la `const` ma non vede effetto, gli serve **ricompilare** (basta tornare in Unity e aspettare). Spiegalo se chiede "perché non funziona".

Livello 2 — Colori e aspetto

Quasi tutto in `Assets/Codice/FabbricaImmagini.cs`.

Domanda	Dove
"Fai Astro blu invece di verde"	<code>CreaAstro()</code> → cambia <code>pelle = new Color(0.40f, 0.85f, 0.40f)</code> in qualcosa tipo <code>new Color(0.30f, 0.50f, 1f)</code>
"Cambia il colore dell'antenna in rosso"	<code>antenna = new Color(1f, 0.85f, 0.20f)</code>
"Cambia lo sfondo da viola scuro a nero"	<code>Avvio.cs:30</code> — <code>c.backgroundColor</code>
"Voglio meno stelle nello sfondo"	<code>CaricatoreLivelli.cs:77</code> — <code>for (int i = 0; i < 120; i++)</code> → cambia il 120
"Cambia il colore della barra energia"	<code>InterfacciaGioco.cs:69</code> — <code>texRiempimentoBarra</code>
"Cuori più grandi"	<code>InterfacciaGioco.cs:164</code> — <code>stileCuori.fontSize</code>

Livello 3 — Logica di gioco semplice

Domanda	Dove
"Quando tocchi un asteroide perdi 2 vite invece di 1"	<code>GestoreGioco.cs:270</code> <code>Vite - 1</code> → <code>Vite - 2</code>
"L'allarme tempo deve partire a 15 secondi, non 10"	<code>InterfacciaGioco.cs:214</code> — <code>gm.TempoRimasto</code> <code><= 10f</code>
"Astro respira più velocemente"	<code>Astro.cs:73</code> — <code>Time.time * 2f</code>
"La caramella deve oscillare di più"	<code>Caramella.cs:52-53</code> — <code>i</code> <code>0.10f</code> e <code>0.12f</code>
"Il cooldown dopo un colpo dura 2 secondi invece di 1"	<code>GestoreGioco.cs:237</code> <code>cooldownDanno = 1.0f</code> → <code>2.0f</code>
"Astro deve essere più grande"	<code>CaricatoreLivelli.cs:146</code> — <code>localScale</code> <code>1.5f</code>

Livello 4 — Modifiche concettuali (qui vede se hai capito davvero)

Domanda	Cosa richiede
"Quando perdi una vita, fai ripartire il tempo da 30 secondi invece che da 60"	In <code>PerdiUnaVita</code> o <code>TempoScaduto</code> in <code>GestoreGioco.cs</code> aggiungere <code>TempoRimasto = 30f;</code>
"Aggiungi un bonus: ogni 5 caramelle, +1 vita"	In <code>SegnalaCaramellaRaccolta</code> controllo <code>if (CaramelleRaccolte % 5 == 0) Vite++;</code>
"Se finisci con più di 20 secondi rimasti, +50 punti bonus"	In <code>SegnalaPortaRaggiunta</code> , prima di <code>MissioneCompletata = true</code>
"Le bombe non devono muoversi"	<code>Bomba.cs:65-69</code> — togliere o commentare il calcolo <code>dx/dy</code>
"La porta si deve aprire anche senza chiave"	<code>GestoreGioco.cs:221</code> — togliere <code>if (!ChiavePresata) return;</code>
"Fai partire il gioco dal livello 2"	<code>GestoreGioco.cs:96</code> — <code>CaricaLivello(0)</code> → <code>CaricaLivello(1)</code>
"Togli il MENU"	<code>InterfacciaGioco.cs:105-109</code> commentare il bottone
"Il flash rosso dello schermo deve essere blu"	<code>InterfacciaGioco.cs:88</code> — il <code>new Color(1f, 0.20f, 0.20f, ...)</code>

Livello 5 — Domande concettuali a voce (dove fai i punti)

1. "Perché usi `static List<Caramella> Attive` invece di cercare le caramelle ogni volta?"

→ **Performance:** `FindObjectByType` è O(n) sull'intera scena ogni frame. Mantenere una lista aggiornata in `OnEnable` / `OnDisable` è O(1).

2. "Cosa succede se in `ControllaCaramelle` itero `for (int i = 0; i < Caramella.Active.Count; i++)` invece che al contrario?"

→ `Raccogli()` chiama `Destroy(gameObject)` che fa `OnDisable` → la caramella si toglie dalla lista mentre ci sto iterando → si saltano elementi o eccezione. Itero al contrario per evitare il problema.

3. "Perché `Impostazioni` è `static class` e non `MonoBehaviour`?"

→ Contiene solo costanti di configurazione, non ha stato, non deve stare nella scena. Una `static class` è più leggera e accessibile da ovunque senza riferimenti.

4. "Perché c'è il `cooldownDanno` ?"

→ Senza, restando "incollato" a un asteroide perderei tutte e 3 le vite in 3 frame consecutivi. Il cooldown di 1 secondo dà il tempo di staccarsi.

5. "Cosa fa `RuntimeInitializeOnLoadMethod` in `Avvio.cs`?"

→ Dice a Unity di chiamare quel metodo automaticamente all'avvio della scena, senza bisogno che lo script sia attaccato a un GameObject. Così la scena può essere vuota.

6. "Perché `Astro.Istanza` è statico?"

→ **Singleton pattern** — c'è un solo Astro in scena, lo rendo accessibile da chiunque senza dover passare riferimenti.

7. "Cos'è `Time.deltaTime` e perché lo moltiplichiamo nei timer?"

→ È il tempo trascorso dall'ultimo frame. Moltiplicandolo (o sottraendolo) i tempi diventano indipendenti dal frame rate.

8. "Perché in `Astro.cs:69` fai `Mathf.Max(Time.deltaTime, 0.000001f)` ?"

→ Per evitare divisione per zero nel calcolo della velocità.

9. "Cosa cambia se metto `Update` o `FixedUpdate` ?"

→ `Update` gira a ogni frame (frame rate variabile), `FixedUpdate` a intervalli fissi (usato per la fisica). Qui non uso fisica Unity quindi `Update` va bene.

10. "Cos'è `sortingOrder` ?"

→ Ordine di disegno degli sprite 2D — chi ha valore più alto sta davanti. Astro ha 10 per stare sempre sopra a tutto.

Domanda "killer" da preparare bene

Quasi sicuro te la fa: "Spiegami il flusso completo da quando premo Play a quando appare Astro."

Risposta strutturata:

1. Unity carica la scena vuota `SceneAvvio.unity`
2. `RuntimeInitializeOnLoadMethod` chiama `Avvio.Inizia()` automaticamente
3. `Avvio` crea la telecamera, un `GestoreGioco` e una `InterfacciaGioco`
4. `GestoreGioco.Start()` chiama `CaricaLivello(0)`
5. `CaricaLivello` legge i dati da `DefinizioneLivelli.Ottieni(0)` (struttura `DatiLivello`)
6. Chiama `CaricatoreLivelli.Carica(dati)` che costruisce stelle, pianeti, **Astro**, asteroidi, bombe, caramelle
7. Da quel momento `Astro.Update()` segue il mouse, `GestoreGioco.Update()` fa scorrere il tempo

Consigli operativi durante l'esame

1. **Tieni aperto Unity** durante l'esame, non solo l'editor — così ogni modifica la *testi al volo* premendo Play.
2. **Apri prima `Impostazioni.cs` e `Avvio.cs`** appena ti dà la domanda: il 70% delle modifiche sta lì o nei file linkati da lì.
3. **Se ti chiede una cosa che non ricordi dove sta**, usa `Cmd+Shift+F` in VS Code per cercare nel progetto (es. cerca `VITE` o `60f` o `Color`).
4. **Se sbagli e Unity dà errore**: non agitarti, leggi il messaggio in console, quasi sempre ti dice file e riga.

Modifiche al progetto - Riepilogo per l'esame

Gioco di riabilitazione "Astro" - Unity 6 / C#

Autore: Filippo Cicirelli

Esame: Sistemi per la Riabilitazione e la Terapia Assistita

Data: 03/06/2026

Questa parte riassume le modifiche fatte al codice del gioco e, per ognuna, come spiegarla e le domande che il professore potrebbe fare.

Struttura dei file e pulizia del progetto

Cosa e' cambiato

- Prima il codice era diviso in 16 file .cs, ora e' raccolto in 6 file dentro Assets/Codice (lo conferma il commit '0520a21 Unifico il codice in 6 file e semplifico gli sprite').
- I 6 file sono: Impostazioni.cs, GestoreGioco.cs, Oggetti.cs, Livelli.cs, FabbricaImmagini.cs, InterfacciaGioco.cs.
- Impostazioni.cs: una sola classe statica con tutti i numeri del gioco (vite, tempo, punti, raggi di tocco), cosi' li cambio in un posto solo.
- GestoreGioco.cs: il 'cervello' del gioco (classe GestoreGioco) piu' la classe Avvio che parte da sola e prepara telecamera e oggetti; tiene punteggio, vite, tempo e le 3 fasi del livello.
- Oggetti.cs: tutte le cose con cui Astro interagisce o che si muovono (Astro, Caramella, Chiave, Porta, Bomba, Asteroide e gli effetti Esplosione, PianetaAmico, Coriandoli).
- Livelli.cs: i dati dei 3 livelli (DefinizioneLivelli) e il codice che li costruisce in scena (CaricatoreLivelli).
- FabbricaImmagini.cs: disegna le immagini (sprite) direttamente dal codice con una legenda lettera=colore, cosi' non servono file immagine esterni.
- InterfacciaGioco.cs: disegna l'HUD a schermo con OnGUI (caramelle, punti, vite, tempo, obiettivo, pulsanti e schermate di vittoria/game over).
- E' stato rimosso lo script di build BuildWebGL.cs insieme alla cartella Editor che lo conteneva: serviva solo per generare automaticamente la versione WebGL, non al gioco.
- E' stato aggiornato il .gitignore aggiungendo .DS_Store, /WebGLBuild/ e /.vscode/ cosi' non finiscono su GitHub.

Come spiegarlo

All'inizio avevo il codice spezzato in tanti file piccoli, sedici in tutto, e per ritrovare una cosa dovevo aprirne troppi. Ho deciso di metterlo in ordine e ora ho solo sei file dentro la cartella Assets/Codice, ognuno con un suo compito chiaro. In Impostazioni.cs ci sono tutti i numeri del gioco, come le vite, il tempo e i punti: cosi' se voglio renderlo piu' facile cambio un valore solo in quel file. In GestoreGioco.cs c'e' il cervello che tiene il punteggio, le vite e decide cosa succede quando prendo una caramella o tocco un asteroide; nello stesso file c'e' anche la parte che fa partire tutto quando premo Play. In Oggetti.cs ci sono tutte le cose del gioco, cioe' il personaggio Astro, le caramelle, la

chiave, la porta, le bombe e gli effetti. In Livelli.cs ci sono i dati dei tre livelli e il pezzo che li costruisce davvero nella scena. FabbricalImmagini.cs disegna le immagini direttamente dal codice, con una legenda dove ogni lettera e' un colore, quindi non mi servono file di immagini. InterfacciaGioco.cs disegna quello che vedo a schermo, cioe' punti, vite, tempo e i pulsanti. Accorpare in sei file aiuta perche' il progetto e' piccolo e cosi' trovo subito le cose, senza saltare tra decine di file. Ho anche tolto un vecchio script di build, BuildWebGL.cs, con la sua cartella Editor: serviva solo a creare la versione per il browser, non al gioco, quindi era solo confusione in piu'. Infine ho sistemato il file .gitignore aggiungendo .DS_Store, la cartella WebGLBuild e la cartella .vscode: il .gitignore e' un elenco di cose che dico a Git di NON salvare su GitHub, perche' sono file del sistema operativo, del mio editor o cartelle generate, e non fanno parte del codice del gioco.

Domande probabili

D: Quanti file C# usa il progetto e quali sono?

R: Sei file dentro Assets/Codice: Impostazioni.cs, GestoreGioco.cs, Oggetti.cs, Livelli.cs, FabbricalImmagini.cs e InterfacciaGioco.cs. Prima erano sedici.

D: Perche' ha messo tutti i numeri del gioco in un file solo, Impostazioni.cs?

R: Perche' cosi' sono tutti centralizzati in un posto: se voglio cambiare le vite, il tempo o i punti modifico un valore in quel file e vale per tutto il gioco, senza cercarlo sparso nel codice.

D: Perche' accorpare in 6 file invece di tenerne 16?

R: Il progetto e' piccolo, quindi 16 file erano troppi e dispersivi. Con 6 file, ognuno con un compito chiaro, trovo le cose piu' in fretta e il codice e' piu' ordinato e leggibile.

D: A cosa serve il file .gitignore e cosa ci ha aggiunto?

R: Il .gitignore e' la lista delle cose che Git non deve salvare su GitHub. Ho aggiunto .DS_Store (file del Mac), /WebGLBuild/ (la cartella generata dalla build) e /.vscode/ (le impostazioni del mio editor): sono file inutili per il codice del gioco.

D: Cosa era BuildWebGL.cs e perche' lo ha rimosso?

R: Era uno script che stava nella cartella Editor e serviva solo a generare in automatico la versione del gioco per il browser (WebGL). Non fa parte del gioco vero e proprio, quindi l'ho rimosso insieme alla sua cartella Editor per tenere il progetto piu' pulito.

La grafica disegnata con le lettere (FabbricalImmagini.cs)

Cosa e' cambiato

- Prima ogni immagine veniva disegnata pixel per pixel con dei calcoli matematici (distanze, `Mathf.Sqrt`) per decidere il colore di ogni puntino: codice difficile da leggere e da cambiare.
- Ora ogni immagine e' un 'disegno' fatto di righe di testo: ogni riga e' una riga di pixel e ogni lettera e' un pixel. La prima riga del testo e' quella in alto.
- C'e' una LEGENDA (un `Dictionary<char, Color>`) che associa ogni lettera a un colore preciso, per esempio 'V' = verde, 'W' = bianco, 'N' = nero.
- Il punto '.' e lo spazio ' ' vogliono dire 'trasparente' (`Color.clear`), cosi' attorno al disegno non c'e' un quadrato colorato.

- I disegni sono griglie 16x16 (costante LATO = 16) e si usano 16 pixel per unita' di Unity (PPU = 16).
- Il metodo Disegna() legge il disegno lettera per lettera e usa la LEGENDA per riempire un array di colori, poi chiama Cuoci() che crea la Texture2D e lo Sprite.
- Per gli oggetti che cambiano colore (caramella, asteroide, pianeta) c'e' DisegnaColorato(): qui '#' = il colore scelto, '+' = una versione piu' scura, 'o' = una versione piu' chiara.
- I metodi Scurisci() e Schiarisci() calcolano in automatico le tonalita' piu' scura e piu' chiara a partire dal colore scelto.
- Risultato: per cambiare un disegno basta cambiare le lettere (per esempio spostare un occhio), senza toccare nessun calcolo matematico.

Come spiegarlo

In questo file creo tutte le immagini del gioco direttamente dal codice, senza usare file immagine esterni. L'idea e' semplice: ogni immagine la scrivo come un disegno fatto di lettere, una riga di testo per ogni riga di pixel. Per esempio per l'alieno Astro ho 16 righe di 16 caratteri ciascuna, quindi una griglia 16x16. In alto, prima della lista dei disegni, ho una LEGENDA: e' un dizionario che dice a quale colore corrisponde ogni lettera. Così 'V' diventa verde, 'W' bianco, 'N' nero, e il punto '.' significa trasparente, cioe' dove non voglio disegnare niente. Quando il gioco mi chiede un'immagine, il metodo Disegna() scorre il disegno lettera per lettera: per ogni lettera guarda nella legenda quale colore usare e lo mette nella casella giusta. Sto attento al fatto che la prima riga di testo deve finire in alto, percio' calcolo la y come LATO - 1 - riga. Alla fine il metodo Cuoci() prende tutti questi colori, li trasforma in una Texture2D e poi in uno Sprite, con il filtro Point così i pixel restano belli a quadretti, in stile pixel art. Per gli oggetti che possono avere colori diversi, come la caramella, l'asteroide e il pianeta, uso un metodo un po' diverso, DisegnaColorato(): qui il carattere '#' prende il colore che gli passo io, '+' prende lo stesso colore ma piu' scuro e 'o' lo prende piu' chiaro. In questo modo con un solo disegno posso fare caramelle di tanti colori diversi, basta che gli dico la tinta. Il vantaggio grande di questo modo di lavorare e' che il codice si capisce subito guardandolo: il disegno dell'alieno sembra davvero un alieno gia' nel testo. E se voglio modificarlo, per esempio cambiare gli occhi o aggiungere un pixel, mi basta cambiare una lettera, senza fare nessun calcolo matematico.

Domande probabili

D: Come fai a disegnare l'alieno Astro?

R: Ho un array di 16 stringhe (ASTRO), una per ogni riga di pixel. Ogni lettera e' un pixel: 'V' verde chiaro, 'v' verde scuro per il bordo, 'W' bianco e 'N' nero per gli occhi, 'b' la pancia chiara. Il metodo Disegna() legge questo testo lettera per lettera, usa la LEGENDA per trovare il colore di ogni lettera e riempie un array di colori, poi Cuoci() lo trasforma in uno Sprite.

D: Perche' il punto '.' significa trasparente?

R: Perche' nella LEGENDA ho associato il carattere '.' (e anche lo spazio ' ') a Color.clear, che e' il colore trasparente. Lo uso negli angoli e attorno alla figura così l'immagine non e' un quadrato pieno, ma si vede solo la sagoma dell'oggetto. Se nel disegno compare una lettera che non e' nella legenda, anche quella diventa trasparente per sicurezza.

D: Come faresti una caramella di un altro colore?

R: La caramella usa il metodo DisegnaColorato(), che prende un colore (tinta) come parametro. Nel disegno CARAMELLA il carattere '#' diventa il colore scelto, '+' la sua versione piu' scura e 'o'

quella piu' chiara. Quindi mi basta chiamare `CreaCaramella()` passando un colore diverso, per esempio `Color.red`, e ottengo la stessa caramella ma rossa. Non devo ridisegnare niente.

D: Come aggiungeresti una nuova immagine al gioco?

R: Prima scrivo un nuovo array di stringhe 16x16 con le lettere giuste, come ho fatto per gli altri disegni. Se uso lettere nuove le aggiungo alla LEGENDA con il loro colore. Poi creo un metodo pubblico, per esempio `CreaStella()`, che chiama `Disegna(STELLA)` (o `DisegnaColorato` se voglio scegliere il colore). Così il resto del gioco può chiedere quella nuova immagine.

D: Perché questo modo è più facile rispetto a disegnare pixel per pixel con i calcoli?

R: Perché prima per ogni pixel facevo calcoli di distanza e `Mathf.Sqrt` per decidere il colore, ed era difficile capire cosa stesse disegnando solo leggendo il codice. Adesso il disegno è fatto di lettere e lo vedo direttamente: l'alieno sembra un alieno già nel testo. Per modificarlo, ad esempio spostare un occhio, cambio una lettera e basta, senza toccare nessuna formula.

Il pannello informazioni / debug (tasto F3)

Cosa è cambiato

- Ho aggiunto un pannello informazioni in stile Minecraft, che si apre e si chiude con il tasto F3 (oppure il tasto backtick ` , `KeyCode.BackQuote`).
- Ho aggiunto due variabili per gli FPS: `'fps'` (media morbida, parte da 60) e `'fpsMinimo'` (il valore più basso visto da quando ho aperto il pannello, parte da 99999).
- Nel metodo `Update` controllo se viene premuto F3: in quel caso inverte il booleano `'debugAperto'` (`debugAperto = !debugAperto`) e azzerò il minimo.
- Sempre in `Update` calcolo gli FPS ogni frame con `fpsOra = 1 / Time.unscaledDeltaTime` e li ammorbidisco con `Mathf.Lerp(fps, fpsOra, 0.1f)`.
- Ho scritto il metodo `DisegnaPannelloDebug`, chiamato in `OnGUI` solo se `debugAperto` è vero, che disegna uno sfondo nero semitrasparente e due colonne di testo.
- Colonna sinistra (dati di gioco): livello, stato, obiettivo, punti, caramelle, vite, tempo rimasto, coordinate X/Y di Astro, velocità, posizione del mouse in pixel, se ha la chiave, bombe attive, e quanti oggetti ci sono in scena (caramelle, asteroidi, bombe).
- Colonna destra (prestazioni e dispositivo): FPS, FPS minimo, lag in ms, target FPS, VSync, `timeScale`; poi info dispositivo via `SystemInfo` (sistema operativo, modello, CPU, core, RAM, scheda video, memoria video, API grafica); poi schermo (risoluzione, refresh, schermo intero); poi versione Unity, piattaforma e tempo di gioco.
- Ho aggiunto un bottone 'INFO (F3)' nell'HUD in basso che fa la stessa cosa del tasto, utile soprattutto nella build WebGL dove il browser può rubare il tasto F3.
- Ho aggiunto uno stile dedicato `'stileDebug'` (testo verdino, piccolo, font 15) creato dentro `CostruisciStili`.
- La colonna destra calcola la sua x con `Mathf.Max(510, Screen.width - 520)` così su schermi stretti, come WebGL a 960x600, le due colonne non si sovrappongono.

Come spiegarlo

Ho voluto un pannello tecnico come quello di Minecraft, che si apre col tasto F3 e mi mostra a colpo d'occhio cosa sta succedendo nel gioco. A sinistra metto i dati della partita: a che livello sono, i punti, le vite, il tempo, dove si trova Astro con le sue coordinate X e Y, la sua velocita', dov'e' il mouse e quanti oggetti ci sono in scena. A destra metto le prestazioni e le informazioni sul computer che sta facendo girare il gioco. La cosa piu' importante e' come calcolo gli FPS, cioe' i fotogrammi al secondo: gli FPS sono uno diviso il tempo passato dall'ultimo frame, cioe' $1 / \text{Time.unscaledDeltaTime}$. Uso `unscaledDeltaTime` e non il normale `deltaTime` perche' io a volte fermo il tempo del gioco (`timeScale` a zero, per esempio quando finisce la partita): se usassi `deltaTime`, in quei momenti farebbe zero e il conto degli FPS impazzirebbe, mentre `unscaledDeltaTime` misura il tempo vero anche col gioco in pausa. Inoltre non mostro il numero secco frame per frame, perche' ballerebbe troppo: lo ammorbidisco con `Mathf.Lerp`, che ad ogni frame avvicina il valore mostrato di un 10 percento verso il valore reale, cosi' il numero resta stabile e leggibile. Il controllo del tasto F3 lo metto in `Update` e non in `OnGUI` per un motivo preciso: `OnGUI` viene chiamato piu' volte per frame (per eventi diversi), quindi se leggesti il tasto li' rischierei di aprire e chiudere il pannello due volte di fila; `Update` invece gira una volta sola per frame ed e' il posto giusto per leggere gli input. Le informazioni sul dispositivo le prendo da `SystemInfo`, che e' una classe di Unity che mi dice com'e' fatto il computer su cui gira il gioco: sistema operativo, processore, RAM, scheda video. Infine ho messo anche un bottone INFO oltre al tasto, perche' nella versione web il browser puo' intercettare F3 (in molti browser apre la ricerca o altro), quindi col bottone l'utente puo' aprire il pannello comunque.

Domande probabili

D: Come calcoli gli FPS e perche' usi `unscaledDeltaTime` invece di `deltaTime`?

R: Gli FPS sono uno diviso il tempo passato dall'ultimo frame: $\text{fpsOra} = 1 / \text{Time.unscaledDeltaTime}$. Uso `unscaledDeltaTime` perche' non risente di `timeScale`: quando fermo o rallento il gioco (`timeScale` a 0), `deltaTime` andrebbe a zero e il calcolo degli FPS impazzirebbe o si bloccherebbe, mentre `unscaledDeltaTime` misura il tempo reale e mi da' sempre gli FPS giusti. Aggiungo anche `Mathf.Max(..., 0.000001f)` per non dividere mai per zero.

D: A cosa serve `Mathf.Lerp` nel calcolo degli FPS?

R: Serve a fare una media morbida. Se mostrassi $1/\text{deltaTime}$ frame per frame il numero salterebbe in continuazione. Con $\text{fps} = \text{Mathf.Lerp}(\text{fps}, \text{fpsOra}, 0.1f)$ ad ogni frame avvicino il valore mostrato del 10 percento verso quello reale, cosi' il numero cambia gradualmente e resta leggibile. E' una specie di filtro che smussa le oscillazioni.

D: Perche' leggi il tasto F3 in `Update` e non dentro `OnGUI`?

R: Perche' `Update` viene chiamato una sola volta per frame ed e' il posto corretto per leggere gli input con `Input.GetKeyDown`. `OnGUI` invece viene chiamato piu' volte nello stesso frame (per i vari eventi della GUI), quindi se leggesti il tasto li' rischierei di invertire il booleano `debugAperto` due volte e il pannello si aprirebbe e chiuderebbe nello stesso istante. In `Update` faccio semplicemente `debugAperto = !debugAperto`.

D: Che cos'e' `SystemInfo` e quali dati ne ricavi?

R: `SystemInfo` e' una classe statica di Unity che da' informazioni sull'hardware e sul sistema dove gira il gioco. Da li' prendo il sistema operativo (`operatingSystem`), il modello del dispositivo, il tipo di CPU (`processorType`) e il numero di core, la RAM in MB (`systemMemorySize`), il nome della scheda video (`graphicsDeviceName`), la memoria video e l'API grafica usata. Mi serve per sapere

su quale macchina sta girando il gioco, utile per il debug e per capire le prestazioni.

D: Perché hai messo anche un bottone INFO se c'è già il tasto F3?

R: Perché il gioco gira anche come build WebGL nel browser, e in molti browser il tasto F3 è già usato (per esempio per la ricerca nella pagina), quindi il browser può catturarlo e il mio gioco non lo riceve. Con il bottone INFO (F3) in basso nell'HUD l'utente può aprire e chiudere il pannello comunque, anche se il tasto non funziona. Il bottone fa esattamente la stessa cosa: inverte debugAperto e azzerà l'FPS minimo.

D: Cosa mostra la colonna sinistra del pannello e da dove prendi quei dati?

R: La colonna sinistra mostra i dati della partita: livello e numero totale di livelli, lo stato (in gioco, missione completata, game over, vittoria finale), l'obiettivo, punti, caramelle, vite, tempo rimasto. Poi i dati di Astro presi da Astro.Istanza: la posizione X e Y, la velocità (Velocita.magnitude) e se ha la chiave. Poi la posizione del mouse in pixel da Input.mousePosition, se ci sono bombe attive, e quanti oggetti ci sono in scena contando le liste statiche Caramella.Active, Asteroide.Tutti e Bomba.Tutte.

D: Cosa indicano lag, target FPS, VSync e timeScale nella colonna destra?

R: Il lag è il tempo che impiega un frame in millisecondi (Time.unscaledDeltaTime per 1000): più è basso, più il gioco è fluido. Il target FPS (Application.targetFrameRate) è il numero di FPS a cui Unity punta. VSync (QualitySettings.vSyncCount) dice se la sincronizzazione verticale è attiva, cioè se i frame vengono allineati al refresh dello schermo. timeScale (Time.timeScale) è la velocità del tempo di gioco: 1 è normale, 0 vuol dire gioco in pausa; lo mostro per ricordare che gli FPS li calcolo col tempo non scalato proprio per non dipendere da questo valore.

Altre domande probabili

Cosa è cambiato

- Accorpamento file: i tanti script separati (Astro, Caramella, Chiave, Porta, Bomba, Asteroide, Avvio, livelli, ecc.) sono stati riuniti in pochi file ordinati per argomento: Oggetti.cs (gli oggetti del gioco), Livelli.cs (definizione e caricamento dei 3 livelli), GestoreGioco.cs (le regole), InterfacciaGioco.cs (l'HUD), FabbricaImmagini.cs (le immagini) e Impostazioni.cs (i numeri da regolare).
- Grafica disegnata a lettere: in FabbricaImmagini.cs ogni immagine è una griglia 16x16 di lettere; una legenda dice che colore corrisponde a ogni lettera (es. V = verde, G = oro, '.' = trasparente). Così non servono file immagine esterni, gli sprite nascono dal codice.
- Tolto lo script di build dall'editor: la cartella Assets/Editor è vuota, non c'è più uno script automatico di build; la build WebGL già pronta resta nella cartella WebGLBuild (index.html + Build).
- Aggiunto il pannello informazioni con il tasto F3 (o il tasto `): mostra a schermo dati di gioco e tecnici (livello, punti, vite, posizione di Astro, FPS, sistema operativo, CPU, RAM). È fatto in OnGUI dentro InterfacciaGioco.cs, in stile schermata di debug di Minecraft.

Come spiegarlo

"In 30 secondi: 'Ho fatto un piccolo gioco in Unity dove guidi un alieno, Astro, muovendolo col mouse: deve raccogliere caramelle, prendere una chiave e portarla alla porta, evitando asteroidi e bombe.

Serve come esercizio di riabilitazione perche' allena la mira e il controllo della mano. Tutto il codice e' mio, in C#, organizzato in pochi file chiari; le immagini sono disegnate dal codice con delle griglie di lettere; il gioco parte da solo con Play e c'e' anche un pannello F3 che mostra i dati tecnici.' Poi mostri il gioco che gira e premi F3."

Domande probabili

D: Perche' hai scelto Unity?

R: Perche' e' un motore gratuito molto usato, con cui posso scrivere in C# e ottenere subito qualcosa che gira a schermo. Mi permette anche di esportare il gioco in WebGL, cioe' farlo girare nel browser senza installare niente.

D: Perche' questo gioco aiuta la riabilitazione?

R: Perche' Astro si muove seguendo il mouse, quindi il paziente deve controllare la mano con precisione per raccogliere le caramelle e portare la chiave alla porta. Ci sono obiettivi semplici, un tempo e delle vite che danno motivazione, e i raggi di raccolta sono regolabili per rendere il gioco piu' facile o piu' difficile.

D: Come parte il gioco senza una scena gia' pronta?

R: Uso l'attributo [RuntimeInitializeOnLoadMethod] nello script Avvio: Unity chiama da sola quel metodo dopo il caricamento della scena. Li' creo la telecamera e i due oggetti principali, GestoreGioco e InterfacciaGioco. Cosi' non devo preparare nessun oggetto a mano nell'editor.

D: Perche' usi il mouse come controllo?

R: Perche' e' immediato e naturale: Astro segue la posizione del mouse. Nel codice prendo Input.mousePosition e con telecamera.ScreenToWorldPoint la trasformo dai pixel dello schermo alle coordinate del mondo di gioco. Per la riabilitazione il mouse e' adatto perche' allena un movimento continuo e mirato.

D: Come faresti ad aggiungere un quarto livello?

R: In Livelli.cs scriverei un metodo CostruisciLivello4 che crea un DatiLivello con titolo, obiettivo, caramelle, asteroidi e bombe, poi lo collegherei nel metodo Ottieni e aumenterei la costante Conteggio da 3 a 4. Il resto del gioco si adatta da solo perche' usa sempre il livello corrente.

D: Cos'e' un MonoBehaviour?

R: E' la classe base di Unity da cui eredito quando voglio che un mio script sia attaccato a un oggetto del gioco. Mi da' i metodi automatici come Awake, Start e Update che Unity chiama al momento giusto. Per esempio Astro e GestoreGioco sono MonoBehaviour.

D: Che differenza c'e' tra Update e OnGUI?

R: Update viene chiamato una volta a ogni frame e lo uso per la logica del gioco, come muovere Astro o far scorrere il tempo. OnGUI serve invece a disegnare l'interfaccia (testi, pulsanti, barre) e puo' essere chiamato piu' volte per frame. Io disegno tutto l'HUD e il pannello F3 dentro OnGUI.

D: Come si fa una build del gioco?

R: Dal menu di Unity in File > Build Settings scelgo la piattaforma, ad esempio WebGL, aggiungo la scena e premo Build: Unity genera la cartella con index.html e i file del gioco. Nel mio progetto la build WebGL e' gia' pronta nella cartella WebGLBuild, quindi posso aprirla nel browser.